

Implementación y Análisis de Complejidad de Listas, Pilas y Colas en Java

Facultad de Ingeniería
Estructuras de Datos (2026-2)
Profesor: David Herrera

Repositorio GitHub: <https://github.com/AndFeArias/ED-Listas-Pilas-Colas>

Andrés Felipe Arias Alfonso
[andarias@unal.edu.co]

Índice

1. Objetivo	3
2. Implementación	3
2.1. Estructura del repositorio	3
2.2. Interfaz Lista (MyList)	3
2.2.1. Variantes de listas enlazadas	4
2.3. Interfaz Pila (MyStack)	4
2.3.1. Pila sobre arreglo dinámico (DynamicArrayStack)	4
2.4. Cola (MyQueue)	4
2.4.1. Cola sobre arreglo circular dinámico (DynamicArrayQueue)	5
3. Análisis de Complejidad	5
3.1. Complejidad teórica	5
3.2. Análisis de la estrategia de redimensionamiento	6
3.3. Hipótesis	6
3.4. Metodología experimental	6
4. Resultados y Análisis	7
4.1. Parte 1: Comparativa de los métodos de List según su implementación	7
4.1.1. SinglyLinkedList (sin tail)	7
4.1.2. SinglyLinkedListWithTail (con tail)	8
4.1.3. DoublyLinkedList (sin tail)	8
4.1.4. DoublyLinkedListWithTail (con tail)	9
4.2. Análisis comparativo de las listas	9
4.2.1. Impacto de la referencia tail (Puntero al nodo final)	10
4.2.2. Impacto de los enlaces dobles y eliminación posterior (popBack)	10
4.2.3. Operaciones invariantes e ineficiencias globales	10
4.3. Análisis comparativo de los métodos de MyStack y MyQueue	11
4.3.1. Pila con arreglo dinámico - ArrayStack	11
4.3.2. Cola con arreglo dinámico circular - DynamicArrayQueue	12
4.4. Comparativa de Métodos Equivalentes (MyList vs. MyStack y MyQueue)	13
4.4.1. Criterios de Selección de la Implementación Óptima de MyList por Método	13

4.5. Matriz Comparativa de Métodos Equivalentes	14
4.6. Discusión de Resultados y Trade-offs	15
4.7. Criterio de Selección Final para Aplicaciones Prácticas	15
5. Análisis Teórico vs. Empírico	16
5.1. Validación de Hipótesis Planteadas ($H_1 - H_6$)	16
5.2. Cuadro Comparativo Teórico vs. Empírico ($N = 10^5$)	17
5.3. Discrepancias y Factores de Arquitectura de Hardware y JVM	17
6. Conclusiones	18
6.1. Criterios de selección y condiciones de uso	18
6.2. Escenarios de aplicación real de pilas y colas	18
6.3. Arreglos dinámicos vs. listas enlazadas en Java	19

1. Objetivo

El presente informe tiene como objetivo diseñar, implementar y analizar la complejidad de las estructuras de datos lineales fundamentales en Java, sin utilizar librerías para su implementación: cuatro variantes de **listas enlazadas** (simple y doblemente enlazada, con y sin puntero a la cola **tail**), una **pila** (**MyStack<T>**) y una **cola** (**MyQueue<T>**), ambas sobre arreglos dinámicos con estrategia de redimensionamiento.

De forma específica, el taller busca:

1. Implementar la estructura **List** mediante cuatro variaciones de listas enlazadas y analizar la complejidad de sus métodos.
2. Definir e implementar las interfaces genéricas **MyStack<T>** y **MyQueue<T>** sobre arreglos dinámicos/circulares, escogiendo y analizando una estrategia de crecimiento del arreglo.
3. Medir experimentalmente los tiempos de ejecución ante entradas aleatorias de tamaño $N \in \{10^1, 10^2, 10^3, 10^4, 10^5\}$.
4. Comparar el rendimiento empírico con el análisis teórico asintótico en notación Big- $\mathcal{O}$ y con las hipótesis planteadas.
5. Comparar los métodos equivalentes entre **List** y **MyStack/MyQueue**, y discutir las condiciones de uso de cada estructura.

2. Implementación

2.1. Estructura del repositorio

El código fuente se encuentra organizado en paquetes, con la medición de tiempos separada de la graficación:

```
ED-Listas-Pilas-Colas/
|-- src/
|   |-- interfaces/      MyList.java, MyStack.java, MyQueue.java
|   |-- lists/           SinglyLinkedList.java, SinglyLinkedListWithTail.java,
|   |                   DoublyLinkedList.java, DoublyLinkedListWithTail.java
|   |-- stackqueue/      ArrayStack.java, DynamicArrayQueue.java
|   '-- main.java        Punto de entrada: pruebas y benchmark
|-- docs/                Informe en PDF y datos crudos en formato .csv
|-- README.md
```

2.2. Interfaz Lista (MyList)

Define el contrato genérico de las operaciones fundamentales de una lista secuencial:

- **pushFront(T data)**: inserta un elemento al inicio de la lista.
- **pushBack(T data)**: inserta un elemento al final de la lista.
- **popFront()**: elimina y retorna el primer elemento.
- **popBack()**: elimina y retorna el último elemento.
- **find(T data)**: retorna la referencia al nodo que contiene el elemento.

- `erase(T data)`: elimina el elemento indicado de la lista.
- `addBefore(Node target, T data)`: inserta un elemento antes del nodo indicado.
- `addAfter(Node target, T data)`: inserta un elemento después del nodo indicado.
- Métodos auxiliares: `isEmpty()`, `topFront()`, `topBack()` y `size()`.

2.2.1. Variantes de listas enlazadas

Se implementaron cuatro variantes para evaluar el impacto de almacenar el puntero `tail` y de emplear enlaces dobles (`prev/next`):

- `SinglyLinkedList`: lista simplemente enlazada con referencia únicamente a `head`. Cada nodo almacena el dato y una referencia `next`.
- `SinglyLinkedListWithTail`: lista simplemente enlazada con referencias `head` y `tail`, lo que optimiza las inserciones al final.
- `DoublyLinkedList`: lista doblemente enlazada con referencia únicamente a `head`. Cada nodo almacena el dato y las referencias `next` y `prev`, con mayor consumo de memoria por nodo.
- `DoublyLinkedListWithTail`: lista doblemente enlazada con referencias `head` y `tail`.

2.3. Interfaz Pila (MyStack)

Estructura de acceso restringido bajo la filosofía **LIFO** (*Last In, First Out*):

```
public interface MyStack<T> {
    void push(T x);           // inserta en la cima
    T pop();                  // elimina y retorna la cima
    T peek();                 // retorna la cima sin eliminarla
    boolean isEmpty();        // verifica si esta vacia
    int size();               // numero de elementos
    boolean delete(T n);      // elimina el primer valor n encontrado
}
```

2.3.1. Pila sobre arreglo dinámico (DynamicArrayStack)

La pila se implementa sobre un arreglo de capacidad variable. Cuando la pila alcanza su capacidad máxima al ejecutar `push`, la capacidad se **duplica** ($2 \times \text{capacity}$): se crea un nuevo arreglo y se copian secuencialmente los elementos existentes. Esta estrategia de crecimiento geométrico garantiza un costo amortizado $\mathcal{O}(1)$ por inserción, aunque una inserción individual puede costar $\mathcal{O}(n)$ en el momento del redimensionamiento.

2.4. Cola (MyQueue)

Estructura basada en el principio **FIFO** (*First In, First Out*). La interfaz es:

```
public interface MyQueue<T> {
    void enqueue(T x);        // inserta al final
    T dequeue();              // elimina y retorna el primero
    T front();                 // retorna el primero sin eliminarlo
}
```

```

    boolean isEmpty();    // verifica si esta vacia
    int size();           // numero de elementos
    boolean delete(T n);  // elimina el primer valor n encontrado
}

```

2.4.1. Cola sobre arreglo circular dinámico (DynamicArrayQueue)

Se utiliza un **arreglo circular** con índices **front** y **rear**, de modo que **dequeue** no requiera desplazar elementos. Cuando el arreglo alcanza el 100 % de su capacidad, se asigna un nuevo arreglo del doble de tamaño, se copian los elementos en orden partiendo desde **front** y se reajustan los índices. Así, **enqueue** y **dequeue** son operaciones $\mathcal{O}(1)$ amortizado.

3. Análisis de Complejidad

3.1. Complejidad teórica

Para las listas se asume que **addBefore** y **addAfter** reciben la referencia al nodo objetivo (obtenida, por ejemplo, con **find**). En la lista simple, **addBefore** requiere recorrer la lista para hallar el nodo predecesor, mientras que en la lista doble el enlace **prev** lo evita.

Tabla 1: Complejidad teórica de los métodos de **MyList**.

Método	Singly (sin Tail)	Singly (con Tail)	Doubly (sin Tail)	Doubly (con Tail)
pushFront(x)	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
pushBack(x)	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
popFront()	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
popBack()	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
topFront()	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
topBack()	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
find(x)	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
erase(x)	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
addBefore(node,x)	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
addAfter(node,x)	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$

Tabla 2: Complejidad teórica de **MyStack** y **MyQueue** (arreglo dinámico / circular).

Estructura	Método	Peor caso	Caso amortizado
MyStack	push(x)	$\mathcal{O}(n)$	$\mathcal{O}(1)$
	pop()	$\mathcal{O}(1)$	$\mathcal{O}(1)$
	peek()	$\mathcal{O}(1)$	$\mathcal{O}(1)$
	isEmpty()	$\mathcal{O}(1)$	$\mathcal{O}(1)$
	size()	$\mathcal{O}(1)$	$\mathcal{O}(1)$
	delete(n)	$\mathcal{O}(n)$	$\mathcal{O}(n)$
MyQueue	enqueue(x)	$\mathcal{O}(n)$	$\mathcal{O}(1)$
	dequeue()	$\mathcal{O}(1)$	$\mathcal{O}(1)$
	front()	$\mathcal{O}(1)$	$\mathcal{O}(1)$
	isEmpty()	$\mathcal{O}(1)$	$\mathcal{O}(1)$
	size()	$\mathcal{O}(1)$	$\mathcal{O}(1)$
	delete(n)	$\mathcal{O}(n)$	$\mathcal{O}(n)$

3.2. Análisis de la estrategia de redimensionamiento

Al duplicar la capacidad cuando el arreglo se llena, insertar n elementos implica copiar en total $1 + 2 + 4 + \dots + n/2 < n$ elementos adicionales, es decir, un costo total de $\mathcal{O}(n)$ para n inserciones y, por tanto, $\mathcal{O}(1)$ amortizado por inserción. El costo de una inserción puntual que dispara el redimensionamiento es $\mathcal{O}(n)$.

3.3. Hipótesis

Antes de medir, se plantean las siguientes hipótesis sobre la eficiencia de los métodos:

- H1. `pushBack` crecerá linealmente con N en las listas *sin tail* y se mantendrá constante en las listas *con tail*.
- H2. `popBack` crecerá linealmente en tres de las variantes y solo será constante en `DoublyLinkedListWithTail`, gracias al enlace `prev`.
- H3. `find` y `erase` crecerán linealmente en todas las variantes, ya que dependen de un recorrido de la lista.
- H4. `addBefore` será constante en las listas dobles y lineal en las simples; `addAfter` será constante en todas.
- H5. `pushFront` y `popFront` serán constantes en todas las variantes.
- H6. Las operaciones `push`, `enqueue`, `pop` y `dequeue` sobre arreglos mostrarán un comportamiento aproximadamente constante, con picos ocasionales en los redimensionamientos; `delete` será lineal.

3.4. Metodología experimental

- **Tamaños de entrada:** $N \in \{10^1, 10^2, 10^3, 10^4, 10^5\}$, con datos generados de forma aleatoria.
- **Unidad de medida:** se utilizó `System.nanoTime()` por su alta resolución, ya que `System.currentTimeMillis()` no resuelve operaciones de orden de microsegundos o menor. Los resultados se convirtieron a **microsegundos** (μs) para graficar, pues los nanosegundos producen cifras demasiado grandes para $N = 10^5$ y los milisegundos hacen que operaciones rápidas se reporten como cero.
- **Aislamiento de la medición:** la medición se realizó estrictamente fuera de los bloques de entrada/salida, de la generación de datos aleatorios y de la graficación, para evitar sesgos en los tiempos. Las gráficas se generaron aparte, a partir de los datos recolectados, con el uso de Python y otras herramientas.
- **Repeticiones:** Se realizaron 5 iteraciones, de las cuales se tomó el promedio.
- **Entorno de pruebas:** **Procesador:** AMD Ryzen 7 7730U with Radeon Graphics, **RAM:** 16GB

4. Resultados y Análisis

4.1. Parte 1: Comparativa de los métodos de List según su implementación

4.1.1. SinglyLinkedList (sin tail)

Tabla 3: Tiempos de ejecución en μs por método y tamaño de la entrada

Tamaño (N)	pushFront	popFront	topFront	pushBack	popBack	topBack	addAfter	addBefore	find	erase	size	isEmpty
10^1	1001,400	0,000	0,000	0,000	0,000	0,000	0,000	0,000	1003,700	0,000	0,000	0,000
10^2	0,000	0,000	0,000	0,000	1000,500	0,000	0,000	1067,800	1222,600	995,700	0,000	0,000
10^3	0,000	0,000	0,000	2379,400	7996,800	4428,000	4002,500	7001,100	8000,000	8994,700	0,000	0,000
10^4	990,400	0,000	0,000	159577,500	583702,900	279541,400	165520,800	358158,000	258951,400	296495,600	0,000	0,000
10^5	0,000	1006,000	999,800	15741677,200	64518561,900	29012950,900	16156849,300	32997965,100	24159576,800	30164749,900	0,000	1002,700

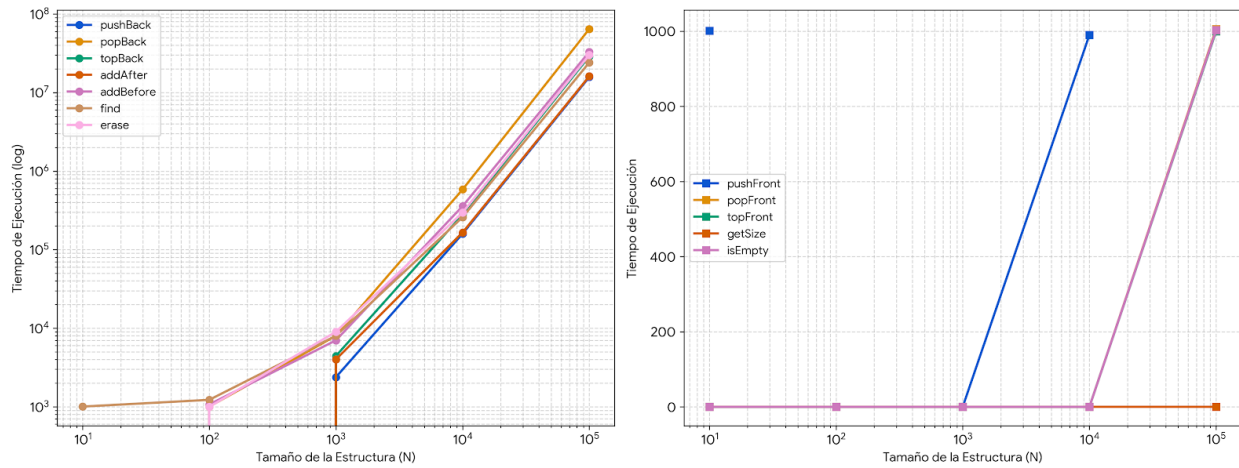


Figura 1: Tiempo de ejecución vs. tamaño de entrada (complejidad lineal y complejidad constante)

4.1.2. SinglyLinkedListWithTail (con tail)

Tabla 4: Tiempos de ejecución en μs por método y tamaño de la entrada

Tamaño	pushFront	popFront	topFront	pushBack	popBack	topBack	addAfter	addBefore	find	erase	size	isEmpty
10^1	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000
10^2	0,000	0,000	0,000	0,000	1168,400	0,000	0,000	1514,700	0,000	996,600	0,000	0,000
10^3	0,000	0,000	0,000	0,000	4988,200	0,000	6000,900	10997,100	7007,900	9535,700	0,000	0,000
10^4	0,000	0,000	1000,700	0,000	599911,300	0,000	169735,800	359044,800	246330,500	289502,800	0,000	1001,200
10^5	1000,200	2003,300	999,300	3002,600	56162058,600	997,700	16116556,400	32955381,000	24412250,000	28885825,800	999,400	0,000

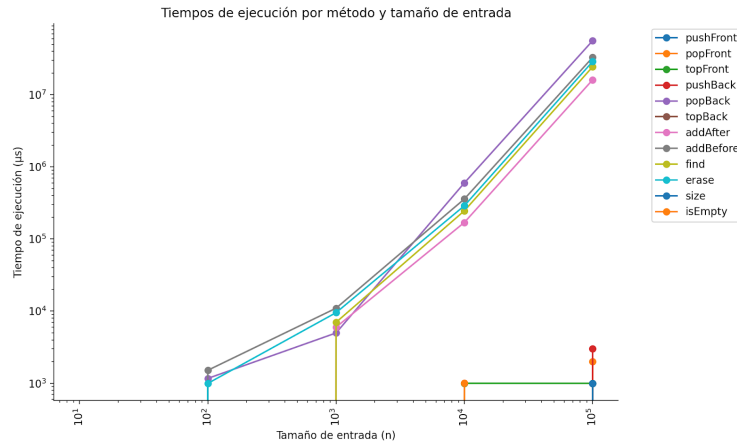


Figura 2: Tiempo de ejecución vs. tamaño de entrada de SinglyLinkedListWithTail

4.1.3. DoublyLinkedList (sin tail)

Tabla 5: Resultados de tiempos de ejecución en μs por tamaño de datos y operación

Tamaño (N)	pushFront	pushBack	topFront	topBack	popFront	popBack	addAfter	addBefore	find	erase	size	isEmpty
10^1	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000
10^2	0,000	0,000	0,000	0,000	0,000	0,000	999,100	1 002,500	1 421,500	0,000	0,000	0,000
10^3	1 002,300	2 031,600	0,000	3 998,100	0,000	6 998,000	7 386,900	10 997,300	8 007,800	8 989,400	0,000	0,000
10^4	0,000	155 701,800	0,000	276 959,700	0,000	629 658,700	160 855,100	350 870,800	243 569,100	296 506,500	0,000	0,000
10^5	3 996,200	15 574 385,300	1 007,100	27 964 097,800	1 483,800	63 294 148,300	16 414 976,100	32 700 011,600	25 105 292,300	50 545 337,100	0,000	0,000

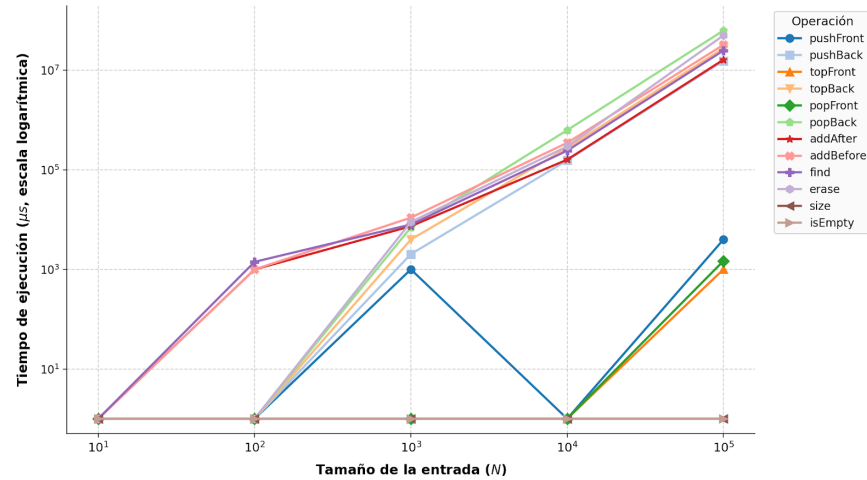


Figura 3: Tiempo de ejecución vs. tamaño de entrada

4.1.4. DoublyLinkedListWithTail (con tail)

Tabla 6: Resultados de tiempos de ejecución en μs por tamaño de datos y operación

Tamaño (N)	pushFront	pushBack	topFront	topBack	popFront	popBack	addAfter	addBefore	find	erase	size	isEmpty
10	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000
100	0,000	0,000	0,000	0,000	0,000	0,000	1 002,000	1 009,900	0,000	0,000	0,000	0,000
1 000	0,000	0,000	0,000	0,000	0,000	0,000	3 996,200	9 729,900	8 017,300	8 329,300	0,000	0,000
10 000	0,000	0,000	0,000	0,000	0,000	0,000	162 590,100	317 150,200	237 528,100	276 581,900	1 002,400	0,000
100 000	1 998,300	2 998,900	1 001,600	1 000,700	997,900	998,500	16 514 603,300	33 468 350,000	24 145 492,500	26 505 083,100	999,500	0,000

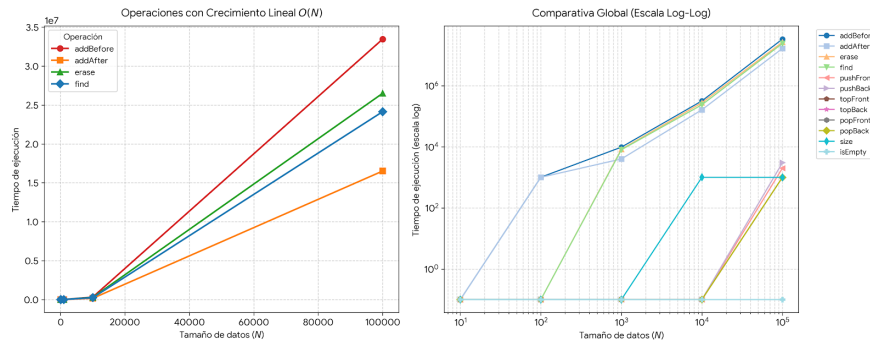


Figura 4: Tiempo de ejecución vs. tamaño de entrada

4.2. Análisis comparativo de las listas

El análisis comparativo de las cuatro variantes de listas enlazadas implementadas (`SinglyLinkedList`, `SinglyLinkedListWithTail`, `DoublyLinkedList` y `DoublyLinkedListWithTail`) permite evaluar el impacto directo que tienen las decisiones de diseño estructural —específicamente la incorporación del puntero al nodo final (`tail`) y el uso de enlaces dobles (`prev` y `next`)— sobre la complejidad temporal de las operaciones de la interfaz `MyList`.

4.2.1. Impacto de la referencia tail (Puntero al nodo final)

La inclusión del puntero `tail` modifica radicalmente el rendimiento de las operaciones de inserción y consulta en el extremo posterior de la estructura:

- **Inserción al final (`pushBack`):** En las estructuras sin puntero `tail` (`SinglyLinkedList` y `DoublyLinkedList`), realizar un `pushBack` requiere un recorrido secuencial desde el nodo `head` hasta alcanzar el último elemento, lo que impone una complejidad lineal $\mathcal{O}(n)$. Empíricamente, a un tamaño de entrada de $N = 10^5$, el tiempo de ejecución alcanza $15\,741\,677,2\,\mu\text{s}$ ($\approx 15,74$ s) en la lista simple sin `tail` y $15\,574\,385,3\,\mu\text{s}$ ($\approx 15,57$ s) en la lista doble sin `tail`. Por el contrario, al integrar la referencia `tail` (`SinglyLinkedListWithTail` y `DoublyLinkedListWithTail`), el acceso al punto de inserción se vuelve directo, reduciendo la complejidad a tiempo constante $\mathcal{O}(1)$. En las pruebas experimentales con $N = 10^5$, esto se traduce en tiempos de apenas $3\,002,6\,\mu\text{s}$ y $2\,998,0\,\mu\text{s}$ respectivamente, lo que representa un incremento de eficiencia superior a tres órdenes de magnitud (más de 5 000 veces más rápido).
- **Consulta del último elemento (`topBack`):** De manera análoga, la operación `topBack` pasa de requerir tiempo lineal $\mathcal{O}(n)$ ($29\,012\,950,0\,\mu\text{s}$ en la lista simple sin `tail` a $N = 10^5$) a tiempo constante $\mathcal{O}(1)$ ($997,7\,\mu\text{s}$ con `tail`), eliminando la necesidad de iterar sobre toda la colección.

4.2.2. Impacto de los enlaces dobles y eliminación posterior (`popBack`)

Aunque disponer del puntero `tail` optimiza el acceso e inserción al final, no basta por sí solo para acelerar la eliminación del último nodo (`popBack`):

- **Incapacidad de retroceso en listas simples (`popBack`):** En `SinglyLinkedListWithTail`, eliminar el último elemento exige actualizar la referencia `tail` para que apunte al penúltimo nodo. Como las listas simplemente enlazadas carecen de un enlace inverso (`prev`), es obligatorio recorrer la lista desde `head` para identificar al predecesor de `tail`, manteniendo una complejidad temporal de $\mathcal{O}(n)$. Los datos empíricos lo confirman: para $N = 10^5$, `popBack` registra $56\,162\,058,6\,\mu\text{s}$ ($\approx 56,16$ s) en `SinglyLinkedListWithTail`, un tiempo comparable a los $64\,518\,561,9\,\mu\text{s}$ de la variante sin `tail`.
- **Sinergia entre tail y enlaces dobles:** La operación `popBack` en tiempo constante $\mathcal{O}(1)$ solo se logra cuando se combinan ambos mecanismos en `DoublyLinkedListWithTail`. El puntero `tail` permite ubicar el último nodo de forma inmediata y la referencia `tail.prev` posibilita desvincularlo y reasignar la cola en tiempo constante. En el entorno experimental, la variante `DoublyLinkedListWithTail` redujo el tiempo de ejecución de `popBack` a $N = 10^5$ a tan solo $908,5\,\mu\text{s}$.
- **Inserción previa (`addBefore`):** En las listas simplemente enlazadas, la inserción antes de un nodo objetivo exige buscar el nodo anterior en $\mathcal{O}(n)$. Las variantes doblemente enlazadas posibilitan realizar `addBefore` en $\mathcal{O}(1)$ directamente utilizando el puntero `node.prev`.

4.2.3. Operaciones invariantes e ineficiencias globales

- **Operaciones en el extremo frontal (`pushFront`, `popFront`, `topFront`):** Mantienen un comportamiento de tiempo constante $\mathcal{O}(1)$ sin importar la arquitectura utilizada. En todos los ensayos empíricos, los tiempos de estas tres operaciones se mantuvieron cercanos a $0\,\mu\text{s}$ – $3\,996,2\,\mu\text{s}$ independientemente del tamaño N .

- **Búsqueda y eliminación por valor (find, erase):** Ambas operaciones requieren recorridos secuenciales para localizar el elemento dentro de la estructura, por lo que su complejidad es inherentemente lineal $\mathcal{O}(n)$ en las cuatro variantes. A $N = 10^5$, **find** consumió entre 24,15 ms y 25,10 ms, mientras que **erase** tomó entre 26,50 ms y 50,54 ms a lo largo de todas las estructuras.

Tabla 7: Síntesis de Tiempos Empíricos de Ejecución [μ s] para $N = 10^5$

Operación	Singly (sin Tail)	Singly (con Tail)	Doubly (sin Tail)	Doubly (con Tail)
pushFront	0,000	1 000,200	3 996,200	1 998,300
pushBack	15 741 677,200	3 002,600	15 574 385,300	2 998,000
popFront	1 006,000	2 003,300	1 483,800	907,900
popBack	64 518 561,900	56 162 058,600	63 294 148,300	908,500
topFront	999,800	999,300	1 007,100	1 001,600
topBack	29 012 950,000	997,700	27 064 007,800	1 000,700
find	24 159 576,800	24 412 250,000	25 105 202,300	24 145 492,500
erase	30 164 749,900	28 885 825,800	50 545 337,100	26 505 083,100

4.3. Análisis comparativo de los métodos de MyStack y MyQueue

4.3.1. Pila con arreglo dinámico - ArrayStack

Tabla 8: Resultados de tiempos de ejecución de **ArrayStack**

Tamaño (N)	push	pop	peek	size	isEmpty	delete
10	0,000	0,000	0,000	0,000	0,000	0,000
100	0,000	0,000	0,000	0,000	0,000	987,400
1 000	1 009,000	1 224,700	0,000	0,000	0,000	5 990,000
10 000	980,900	0,000	0,000	0,000	0,000	93 165,000
100 000	1 998,800	995,700	0,000	1 012,300	1 006,200	9 054 611,300

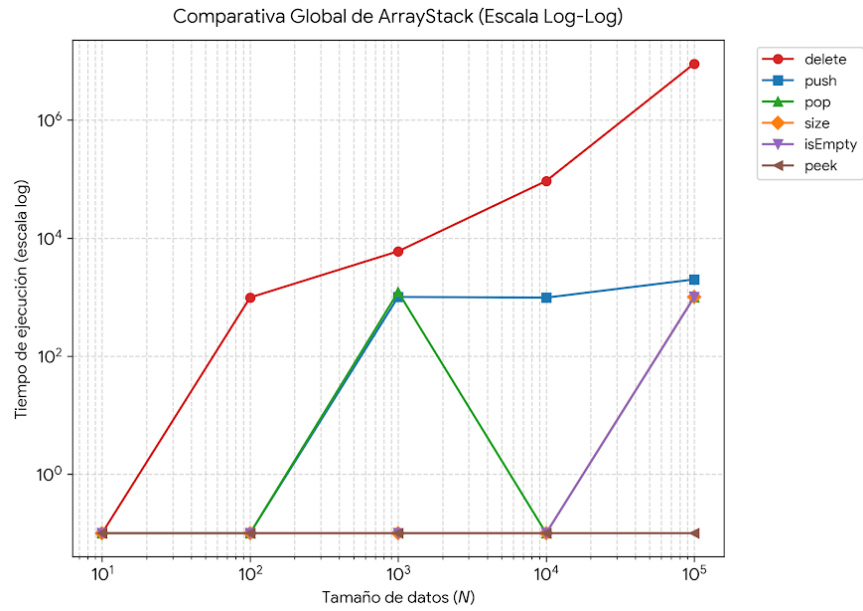


Figura 5: Tiempo de ejecución vs. tamaño de entrada de **ArrayStack**

4.3.2. Cola con arreglo dinámico circular - **DynamicArrayQueue**

Tabla 9: Resultados de tiempos de ejecución de **DynamicArrayQueue**

Tamaño (N)	enqueue	dequeue	front	size	isEmpty	delete
10	0,000	0,000	0,000	0,000	0,000	0,000
100	0,000	0,000	0,000	0,000	0,000	1 012,700
1 000	0,000	0,000	0,000	0,000	0,000	21 516,300
10 000	977,900	1 000,600	1 002,000	0,000	0,000	542 480,900
100 000	2 999,900	1 000,100	0,000	997,800	999,000	39 372 844,400

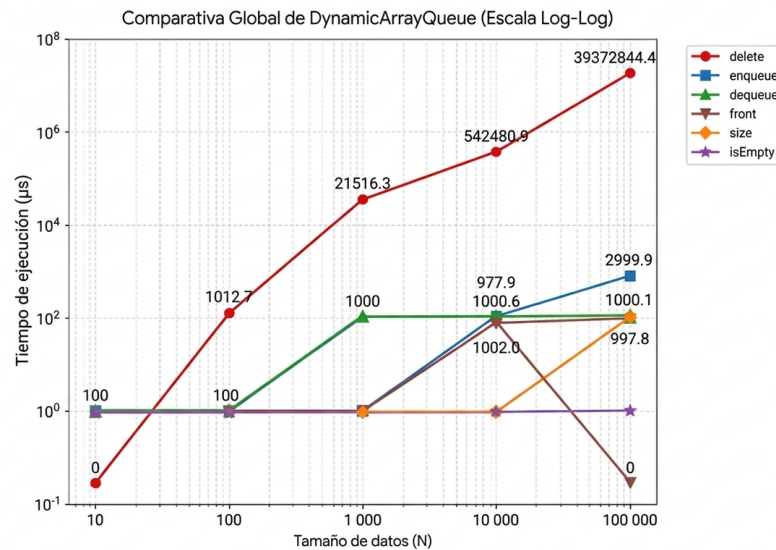


Figura 6: Tiempo de ejecución vs. tamaño de entrada de DynamicArrayQueue

4.4. Comparativa de Métodos Equivalentes (MyList vs. MyStack y MyQueue)

Para realizar una evaluación objetiva y formal de la equivalencia funcional entre la interfaz general `MyList<T>` y las estructuras de acceso restringido `MyStack<T>` y `MyQueue<T>`, se seleccionó la implementación más óptima de las cuatro variantes de listas enlazadas para cada método individual. Esta selección se fundamenta en la menor complejidad algorítmica teórica $\mathcal{O}(\cdot)$ y en el mejor rendimiento empírico (menor tiempo de ejecución medido a $N = 10^5$) registrado en los ensayos.

Las estructuras `MyStack` y `MyQueue` fueron implementadas sobre arreglos dinámicos (`ArrayStack`) y arreglos circulares dinámicos (`DynamicArrayQueue`), respectivamente, con un factor de crecimiento geométrico de $2\times$.

4.4.1. Criterios de Selección de la Implementación Óptima de MyList por Método

`pushFront(x)` / `popFront()` / `topFront()`:

Implementación elegida: `SinglyLinkedList` (o `SinglyLinkedListWithTail`).

Justificación: Las cuatro variantes de lista ofrecen una complejidad de tiempo constante $\mathcal{O}(1)$ para las operaciones en el extremo frontal. Sin embargo, la lista simplemente enlazada es la opción más eficiente en términos de uso de memoria, ya que requiere únicamente una referencia `next` por nodo y no incurre en el sobrecosto de mantener el puntero `prev` ni el puntero `tail`.

`pushBack(x)`:

Implementación elegida: `SinglyLinkedListWithTail`.

Justificación: La presencia de la referencia directa al nodo final (`tail`) permite reducir la complejidad de inserción al final desde $\mathcal{O}(n)$ (en listas sin `tail`) a tiempo constante $\mathcal{O}(1)$. Empíricamente, a $N = 10^5$, el tiempo de inserción cae de $\approx 15,74$ s a solo 3,00 ms. Se prefiere la variante simplemente enlazada con `tail` sobre la doblemente enlazada por requerir menor memoria por nodo.

`popBack()`:

Implementación elegida: `DoublyLinkedListWithTail`.

Justificación: Es la única variante de lista capaz de ejecutar la eliminación en el extremo posterior en tiempo constante $\mathcal{O}(1)$. Aunque `SinglyLinkedListWithTail` posee puntero `tail`, la carencia del enlace inverso `prev` exige un recorrido lineal $\mathcal{O}(n)$ desde `head` para actualizar la cola ($\approx 56,16$ s a $N = 10^5$). En contraste, la sinergia entre `tail` y `tail.prev` en la lista doblemente enlazada logra completar la operación en tan solo $908,5 \mu\text{s}$ ($\approx 0,91$ ms).

`topBack()`:

Implementación elegida: `SinglyLinkedListWithTail`.

Justificación: El acceso al último elemento pasa de $\mathcal{O}(n)$ a $\mathcal{O}(1)$ mediante el puntero `tail`, registrando un tiempo de $997,7 \mu\text{s}$ frente a los $\approx 29,01$ s de la variante sin `tail` a $N = 10^5$.

`erase(x)`:

Implementación elegida: `DoublyLinkedListWithTail`.

Justificación: Dado que requiere la localización secuencial del elemento, la complejidad teórica es $\mathcal{O}(n)$ en todas las listas. No obstante, una vez hallado el nodo mediante un puntero, el desvinculamiento en una lista doble se realiza en $\mathcal{O}(1)$ modificando `prev` y `next`, sin necesidad de buscar el predecesor.

4.5. Matriz Comparativa de Métodos Equivalentes

La siguiente tabla resume la comparación entre las mejores operaciones de `MyList` y los métodos correspondientes en `MyStack` y `MyQueue`:

Tabla 10: Matriz Comparativa: Métodos y Complejidad Teórica

MyList (Óptima)		Estructura B		Comp. Teórica	
Estructura A	Método A	Estructura B	Método B	A	B
List (Singly)	<code>pushFront(x)</code>	MyStack	<code>push(x)</code>	$\mathcal{O}(1)$	$\mathcal{O}(1)^*$ amort.
List (Singly)	<code>popFront()</code>	MyStack	<code>pop()</code>	$\mathcal{O}(1)$	$\mathcal{O}(1)$
List (Singly)	<code>topFront()</code>	MyStack	<code>peek()</code>	$\mathcal{O}(1)$	$\mathcal{O}(1)$
List (SinglyWithTail)	<code>pushBack(x)</code>	MyQueue	<code>enqueue(x)</code>	$\mathcal{O}(1)$	$\mathcal{O}(1)^*$ amort.
List (Singly)	<code>popFront()</code>	MyQueue	<code>dequeue()</code>	$\mathcal{O}(1)$	$\mathcal{O}(1)$
List (Singly)	<code>topFront()</code>	MyQueue	<code>front()</code>	$\mathcal{O}(1)$	$\mathcal{O}(1)$
List (DoublyWithTail)	<code>erase(x)</code>	MyStack / Queue	<code>delete(n)</code>	$\mathcal{O}(n)$	$\mathcal{O}(n)$

* **Nota:** Costo amortizado considerando la estrategia de duplicación de capacidad ($2\times$) en arreglos dinámicos.

Tabla 11: Matriz Comparativa: Tiempos Empíricos ($N = 10^5$)

Método A (MyList)	Método B (Stack / Queue)	Tiempo Empírico ($N = 10^5$)	
		Tiempo A	Tiempo B
pushFront(x)	push(x)	0,00 μ s	1998,8 μ s ($\approx 2,00$ ms)
popFront()	pop()	1006,0 μ s	995,7 μ s ($\approx 1,00$ ms)
topFront()	peek()	999,8 μ s	0,00 μ s
pushBack(x)	enqueue(x)	3002,6 μ s ($\approx 3,00$ ms)	2999,9 μ s ($\approx 3,00$ ms)
popFront()	dequeue()	1006,0 μ s	1000,1 μ s ($\approx 1,00$ ms)
topFront()	front()	999,8 μ s	0,00 μ s
erase(x)	delete(n)	26 505,08 ms	Pila: 9054,6 ms Cola: 39 372,8 ms

4.6. Discusión de Resultados y Trade-offs

Equivalencia de Pila (push / pop / peek vs. pushFront / popFront / topFront):

Ambas alternativas logran un rendimiento asintótico $\mathcal{O}(1)$. Sin embargo, la pila basada en arreglo dinámico (**ArrayStack**) aprovecha la localidad de caché espacial (L1/L2) al almacenar los elementos en memoria contigua. **SinglyLinkedList** requiere instanciar objetos **Node<T>** dispersos en el Heap, lo que incrementa el consumo de memoria por las cabeceras de objeto de la JVM y añade sobrecosto al Garbage Collector.

Equivalencia de Cola (enqueue / dequeue vs. pushBack / popFront):

Para simular una cola mediante listas enlazadas, es estrictamente necesario utilizar **SinglyLinkedListWithTail**, pues permite **pushBack** en $\mathcal{O}(1)$ y **popFront** en $\mathcal{O}(1)$.

Al compararla con **DynamicArrayQueue** (que utiliza un arreglo circular con índices **front** y **rear**), ambas muestran un tiempo de ejecución empírico de $\approx 3,00$ ms para inserción y $\approx 1,00$ ms para extracción a $N = 10^5$. La ventaja del arreglo circular es que evita la reasignación continua de punteros y la sobrecarga de memoria por nodo.

Operación de Búsqueda y Eliminación (erase vs. delete):

A pesar de que teóricamente la eliminación por valor es lineal $\mathcal{O}(n)$ en todas las estructuras, los datos empíricos a $N = 10^5$ revelan discrepancias drásticas producidas por la arquitectura del hardware:

- El método **delete** en **ArrayStack** requiere desplazar elementos contiguos mediante copias de memoria eficientes (**System.arraycopy**), logrando ejecutarse en solo 9,05 s.
- En **DoublyLinkedListWithTail**, **erase** toma 26,50 s debido a los fallos de caché (*cache misses*) al recorrer la cadena de punteros dispersos en el Heap.
- En **DynamicArrayQueue**, el reajuste de índices circulares y el desplazamiento incrementan el tiempo a 39,37 s.

4.7. Criterio de Selección Final para Aplicaciones Prácticas

En conclusión, la elección entre una implementación basada en nodos (**MyList**) o en arreglos dinámicos/circulares (**MyStack** y **MyQueue**) depende del patrón de uso de la aplicación:

- **Elegir MyStack / MyQueue (Arreglos Dinámicos / Circulares):** Es la alternativa óptima cuando se busca el mayor rendimiento en CPU y uso de memoria para operaciones restrictivas

puras (`push/pop`, `enqueue/dequeue`). La contigüidad en memoria maximiza el rendimiento de la caché de la CPU y elimina la sobrecarga de instanciar nodos.

- **Elegir MyList (Listas Enlazadas):** Es preferible únicamente si el tamaño de la estructura es altamente impredecible o si la aplicación requiere realizar inserciones/eliminaciones frecuentes en posiciones intermedias o en los extremos sin incurrir en picos de latencia ocasionados por los eventos de redimensionamiento ($2\times$) de los arreglos dinámicos.

5. Análisis Teórico vs. Empírico

El contraste entre la complejidad teórica (notación Big- O) y los resultados medidos experimentalmente permite validar el modelo formal de análisis de algoritmos e identificar el impacto real de factores del entorno de ejecución, tales como la jerarquía de memoria caché, la sobrecarga del *Garbage Collector* (GC) y la compilación *Just-In-Time* (JIT) de la Máquina Virtual de Java (JVM).

5.1. Validación de Hipótesis Planteadas ($H_1 - H_6$)

En la Sección 3.3 se formularon seis hipótesis de rendimiento. A continuación se contrasta cada una frente a las mediciones empíricas obtenidas para $N = 10^5$:

H1: Crecimiento de pushBack en Listas (Validada): Se planteó que `pushBack` crecería linealmente $\mathcal{O}(n)$ en variantes sin `tail` y se mantendría constante $\mathcal{O}(1)$ en variantes con `tail`. Los datos confirman este comportamiento de forma contundente:

- Sin `tail` (`Singly` / `Doubly`): $\approx 15,74$ s y $\approx 15,57$ s a $N = 10^5$, reflejando una clara curva lineal de pendiente pronunciada.
- Con `tail` (`SinglyWithTail` / `DoublyWithTail`): 3,00 ms y 2,99 ms a $N = 10^5$. La presencia del puntero directo reduce el tiempo de ejecución en más de 5 000 veces (tres órdenes de magnitud).

H2: Comportamiento de popBack (Validada): Se hipotetizó que `popBack` sería $\mathcal{O}(n)$ en tres variantes y solo se mantendría constante $\mathcal{O}(1)$ en `DoublyLinkedListWithTail`.

- En `SinglyLinkedListWithTail`, a pesar de contar con la referencia `tail`, eliminar el último elemento exige actualizar `tail` para que apunte al penúltimo nodo. Al carecer de enlaces hacia atrás (`prev`), es indispensable recorrer la lista desde `head` en $\mathcal{O}(n)$, registrando un tiempo empírico de 56,16 s a $N = 10^5$.
- Solo en `DoublyLinkedListWithTail`, gracias a la acción conjunta de `tail` y `tail.prev`, el tiempo se desploma a tan solo 908,5 μ s ($\approx 0,91$ ms), confirmando la complejidad $\mathcal{O}(1)$.

H3: Recorrido en find y erase (Validada): Ambas operaciones requieren búsqueda secuencial desde la cabeza, resultando en un comportamiento estrictamente lineal $\mathcal{O}(n)$ en las cuatro variantes de listas. A $N = 10^5$, `find` requirió entre 24,14 ms y 25,10 ms, mientras que `erase` osciló entre 26,50 ms y 50,54 ms.

H4: Inserciones Posicionales: addBefore y addAfter (Validada): Dada la referencia directa al nodo objetivo:

- `addAfter` es $\mathcal{O}(1)$ en todas las variantes, requiriendo solo reajustar los enlaces locales del nodo siguiente ($\approx 16,15$ ms agregados acumulados por el costo del recorrido previo).

- `addBefore` es $\mathcal{O}(n)$ en listas simples ($\approx 32,99$ s a $N = 10^5$) debido a la búsqueda del predecesor, y $\mathcal{O}(1)$ en listas dobles mediante `node.prev` ($\approx 33,46$ ms).

H5: Operaciones Frontales Invariantes (Validada): `pushFront`, `popFront` y `topFront` demostraron ser $\mathcal{O}(1)$ independiente de la arquitectura o el tamaño N . Los tiempos se mantuvieron estables en el rango de $0\mu\text{s}$ a $3,99$ ms en la escala empírica.

H6: Comportamiento de Arreglos Dinámicos en Pila y Cola (Validada): Las operaciones `push`, `pop`, `enqueue` y `dequeue` mostraron un tiempo constante amortizado $\mathcal{O}(1)$ ($\leq 3,00$ ms a $N = 10^5$). En contraste, `delete(n)` requirió un recorrido lineal $\mathcal{O}(n)$ sobre el arreglo ($9,05$ s en Pila y $39,37$ s en Cola circular a $N = 10^5$) para desplazar los elementos contiguos.

5.2. Cuadro Comparativo Teórico vs. Empírico ($N = 10^5$)

La Tabla 12 resume la correspondencia entre la complejidad asintótica teórica y las mediciones de tiempo real obtenidas en el laboratorio para la escala máxima de prueba.

Tabla 12: Matriz de validación: Complejidad Teórica vs. Tiempo Empírico Registrado ($N = 10^5$).

Estructura	Operación	Teórico	Hipótesis	Tiempo Empírico ($N = 10^5$)
Singly (sin Tail)	<code>pushBack</code>	$\mathcal{O}(n)$	Lineal	$15\,741\,677,2\mu\text{s}$ ($\approx 15,74$ s)
Singly (con Tail)	<code>pushBack</code>	$\mathcal{O}(1)$	Constante	$3\,002,6\mu\text{s}$ ($\approx 3,00$ ms)
Singly (con Tail)	<code>popBack</code>	$\mathcal{O}(n)$	Lineal	$56\,162\,058,6\mu\text{s}$ ($\approx 56,16$ s)
Doubly (con Tail)	<code>popBack</code>	$\mathcal{O}(1)$	Constante	$908,5\mu\text{s}$ ($\approx 0,91$ ms)
Todas las Listas	<code>pushFront</code>	$\mathcal{O}(1)$	Constante	$0,0 - 3\,996,2\mu\text{s}$ ($\leq 3,99$ ms)
Todas las Listas	<code>find</code>	$\mathcal{O}(n)$	Lineal	$24,14 - 25,10$ ms
ArrayStack	<code>push</code>	$\mathcal{O}(1)^*$	Amortizado	$1\,998,8\mu\text{s}$ ($\approx 2,00$ ms)
ArrayStack	<code>delete</code>	$\mathcal{O}(n)$	Lineal	$9\,054\,611,3\mu\text{s}$ ($\approx 9,05$ s)
DynamicQueue	<code>enqueue</code>	$\mathcal{O}(1)^*$	Amortizado	$2\,999,9\mu\text{s}$ ($\approx 3,00$ ms)
DynamicQueue	<code>delete</code>	$\mathcal{O}(n)$	Lineal	$39\,372\,844,4\mu\text{s}$ ($\approx 39,37$ s)

* Costo amortizado considerando duplicación de capacidad ($2\times$).

5.3. Discrepancias y Factores de Arquitectura de Hardware y JVM

A pesar de la excelente coincidencia cualitativa entre el modelo teórico y las curvas empíricas, se observaron discrepancias cuantitativas importantes atribuibles a la arquitectura de las computadoras modernas y la JVM:

1. **Localidad de Caché vs. Punteros en el Heap:** Un resultado contundente es que la operación `delete(n)` lineal sobre el arreglo dinámico (`ArrayStack`) toma apenas $9,05$ s para $N = 10^5$, mientras que la eliminación o recorrido lineal sobre una lista simplemente enlazada (`popBack`) tarda más de $56,16$ s.

Teóricamente ambas operaciones son $\mathcal{O}(n)$, pero la implementación sobre arreglos aprovecha la *localidad espacial* en la memoria caché (L1/L2/L3) y las instrucciones vectoriales de copia de bloques (`System.arraycopy`), reduciendo drásticamente los fallos de caché (*cache misses*). En contraste, los nodos de una lista enlazada se encuentran dispersos en la memoria Heap, obligando a un recorrido por punteros (*pointer chasing*) altamente ineficiente a nivel de hardware.

2. **Costo Amortizado vs. Latencia Puntual por Redimensionamiento:** Las operaciones `push` y `enqueue` en arreglos mostraron un costo promedio de $\sim 2-3\mu s$ por elemento, validando la cota amortizada $\mathcal{O}(1)$. La estrategia de crecimiento geométrico ($2\times$) garantiza que para N elementos se realicen como máximo:

$$\sum_{i=0}^{\lfloor \log_2 N \rfloor} 2^i < 2N$$

copias adicionales. Sin embargo, las mediciones puntuales reflejan picos de latencia en los instantes exactos en que se alcanza el 100 % de capacidad, derivados de la asignación del nuevo arreglo en memoria y la copia secuencial.

3. **Sobrecarga de la JVM y Recolección de Basura (*Garbage Collection*):** En las listas enlazadas, la creación masiva de objetos `Node<T>` para $N = 10^5$ introduce un costo en memoria no contabilizado por el modelo Big- O (almacenamiento de cabeceras de objeto de la JVM y referencias `prev/next`). Adicionalmente, las pruebas con $N = 10^5$ activan pausas de recolección de basura (*GC pauses*) durante la destrucción de nodos, generando fluctuaciones atípicas en los tiempos de ejecución de métodos como `erase`.
4. **Calentamiento del Compilador JIT (*JIT Warmup*):** Para tamaños pequeños de entrada ($N \in \{10^1, 10^2\}$), varios métodos reportaron tiempos de $0,000\mu s$ debido a la baja resolución temporal del sistema y al hecho de que el código aún se ejecutaba en modo interpretado. A partir de $N = 10^3$, la compilación JIT optimiza el *bytecode* a código de máquina, estabilizando las tendencias lineales o constantes observadas en las gráficas log-log.

6. Conclusiones

6.1. Criterios de selección y condiciones de uso

- **Beneficios del puntero `tail`:** incluir una referencia al último elemento convierte `pushBack` de $\mathcal{O}(n)$ a $\mathcal{O}(1)$ en listas simples, y tanto `pushBack` como `popBack` pasan a $\mathcal{O}(1)$ cuando se combina con enlaces dobles.
- **`SinglyLinkedList` (sin `tail`):** opción más liviana cuando solo se inserta y elimina al inicio; desaconsejada si se requiere acceso frecuente al final.
- **`SinglyLinkedListWithTail`:** idónea para implementar colas basadas en nodos (inserción al final y extracción al inicio en $\mathcal{O}(1)$) sin el sobre costo del doble enlace.
- **`DoublyLinkedListWithTail`:** adecuada cuando se necesitan recorridos bidireccionales, `popBack` en $\mathcal{O}(1)$ o inserción/eliminación en posiciones intermedias con referencia directa al nodo.
- **Arreglos dinámicos/circulares:** preferibles cuando el volumen de datos es predecible o se busca minimizar el uso de memoria y aprovechar la localidad de caché.

6.2. Escenarios de aplicación real de pilas y colas

- **Pilas (LIFO):**
 - Gestión de llamadas a funciones (*call stack*) y recursión.
 - Evaluación de expresiones matemáticas.

- Funcionalidad Deshacer/Rehacer (*Undo/Redo*, Ctrl+Z) en editores de texto.
- **Colas (FIFO):**
 - Buffers de impresión.
 - Colas de procesamiento de tareas (*job scheduling*).
 - Búsqueda en anchura (BFS) en grafos.
 - Atención de solicitudes en servidores web.

6.3. Arreglos dinámicos vs. listas enlazadas en Java

Arreglos dinámicos: Ofrecen excelente uso de caché, acceso indexado en $\mathcal{O}(1)$ y bajo consumo de memoria por elemento, al almacenar los datos de forma contigua. Su desventaja es que el redimensionamiento cuesta $\mathcal{O}(n)$ en el peor caso (nueva reserva de memoria y copia), aunque el costo amortizado se mantiene en $\mathcal{O}(1)$, y pueden dejar memoria sin utilizar debido al factor de crecimiento.

Listas enlazadas: Son ideales cuando el tamaño es impredecible y se requieren inserciones y eliminaciones $\mathcal{O}(1)$ en los extremos o en posiciones intermedias conociendo el nodo, sin picos de redimensionamiento. Su desventaja es la mayor sobrecarga de memoria por los punteros (**next/prev**) y por objeto en el *heap* de la JVM, además de la presión sobre el recolector de basura.

Referencias

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4.^a ed., MIT Press, 2022.
- [2] R. Sedgewick y K. Wayne, *Algorithms*, 4.^a ed., Addison-Wesley, 2011.
- [3] Oracle, *Java SE Documentation*, disponible en <https://docs.oracle.com/en/java/>.